Upside Dine: Detailed Deployment Implementation Guide

This guide explains **exactly** how your env variables, Dockerfiles, Docker Compose files, and containers will be managed across the distributed microservice architecture.

Architecture Recap

Service	Platform	What Runs There
Frontend	**Netlify**	React/Vite static build
Backend API	**Render** (Node 1) + **Koyeb** (Node 2)	Django REST API
Database	**Supabase**	Managed PostgreSQL
Cache/Broker	**Upstash**	Managed Redis
ML Service	**IITK Server**	FastAPI + YOLOv8
Celery Workers	**IITK Server**	Celery Worker + Beat
Load Balancer	**IITK Server** (Nginx)	Routes `/api` traffic to Render & Koyeb
WebSockets	**IITK Server**	Daphne (Django Channels)

Part 1: External Services Setup

1.1 Supabase (PostgreSQL Database)

1. Go to supabase.com, create a free project.
2. Navigate to **Settings → Database** and copy your connection string. It will look like:

```
postgresql://postgres.[ref]:[password]@aws-0-
[region].pooler.supabase.com:6543/postgres
```

3. Extract these values for your `.env` :

```
DATABASE_NAME=postgres
DATABASE_USER=postgres.[your-ref]
DATABASE_PASSWORD=your-supabase-password
DATABASE_HOST=aws-0-ap-south-1.pooler.supabase.com
DATABASE_PORT=6543
```

1.2 Upstash (Serverless Redis)

1. Go to upstash.com, create a free Redis database.
2. Copy the connection URL from the dashboard. It looks like:

```
rediss://default:AxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxQ@healthy-panda-
```

```
12345.upstash.io:6379
```

3. This single URL replaces ALL your Redis config:

```
REDIS_URL=rediss://default:AxxxxQ@healthy-panda-12345.upstash.io:6379
CELERY_BROKER_URL=rediss://default:AxxxxQ@healthy-panda-
12345.upstash.io:6379/1
CELERY_RESULT_BACKEND=rediss://default:AxxxxQ@healthy-panda-
12345.upstash.io:6379/2
```

> ***Note:*** *Upstash uses* `rediss://` *(with double s) for TLS encryption, not* `redis://` *.*

Part 2: Environment Variables Per Platform

Current `backend/.env` (LOCAL — what you have now)

Your current `.env` points to Docker container hostnames (`db` , `redis` , `ml_service`). These **only work locally** inside Docker Compose's internal network. For production, each platform needs its own set of values.

2.1 Render Environment Variables (Backend API)

Set these in Render's **Dashboard → Environment tab** (NOT in a file — Render injects them at runtime):

```
# Django
SECRET_KEY=<generate-a-strong-random-key>
DEBUG=False
ALLOWED_HOSTS=upside-dine-api.onrender.com,your-koyeb-url.koyeb.app,your-iitk-ip
ENVIRONMENT=production

# Supabase PostgreSQL
DATABASE_NAME=postgres
DATABASE_USER=postgres.[your-ref]
DATABASE_PASSWORD=<supabase-password>
DATABASE_HOST=aws-0-ap-south-1.pooler.supabase.com
DATABASE_PORT=6543

# Upstash Redis
REDIS_URL=rediss://default:xxxx@your-db.upstash.io:6379/0
CELERY_BROKER_URL=rediss://default:xxxx@your-db.upstash.io:6379/1
CELERY_RESULT_BACKEND=rediss://default:xxxx@your-db.upstash.io:6379/2

# Email (same as before)
EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend
EMAIL_HOST=smtp-relay.brevo.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=<your-brevo-user>
EMAIL_HOST_PASSWORD=<your-brevo-password>
DEFAULT_FROM_EMAIL=Upside Dine <upsidedine9@gmail.com>
```

```
# JWT
JWT_SECRET_KEY=<generate-a-strong-random-key>
JWT_ACCESS_TOKEN_LIFETIME=30
JWT_REFRESH_TOKEN_LIFETIME=7

# CORS — allow Netlify frontend
CORS_ALLOWED_ORIGINS=https://upside-dine.netlify.app

# ML Service on IITK
ML_SERVICE_URL=http://<iitk-server-public-ip>:8002

# WebSocket Channels (Upstash Redis)
CHANNEL_LAYERS_BACKEND=channels_redis.core.RedisChannelLayer
CHANNEL_LAYERS_HOST=your-db.upstash.io
CHANNEL_LAYERS_PORT=6379

# Security
SECURE_SSL_REDIRECT=True
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SECURE=True
```

2.2 Koyeb Environment Variables (Backend API Node 2)

Exact same variables as Render above. Just update `ALLOWED_HOSTS` to include the Koyeb URL.

2.3 Netlify Environment Variables (Frontend)

Set these in **Netlify → Site Settings → Environment Variables**:

```
VITE_API_URL=http://<iitk-server-public-ip>
VITE_WS_URL=wss://<iitk-server-public-ip>/ws
VITE_ENV=production
```

> *The `VITE_API_URL` points to the **IITK Nginx load balancer**, which then distributes traffic to Render and Koyeb.*

2.4 IITK Server `.env` (ML + Celery + Channels + Nginx)

This is a `.env` file that lives on the IITK server:

```
# Upstash Redis (ML service pushes results here)
REDIS_URL=rediss://default:xxxx@your-db.upstash.io:6379/0
REDIS_HOST=your-db.upstash.io
REDIS_PORT=6379

# Celery (connects to same Upstash Redis)
CELERY_BROKER_URL=rediss://default:xxxx@your-db.upstash.io:6379/1
CELERY_RESULT_BACKEND=rediss://default:xxxx@your-db.upstash.io:6379/2

# Supabase DB (for Celery tasks that need DB access)
```

```
DATABASE_NAME=postgres
DATABASE_USER=postgres.[your-ref]
DATABASE_PASSWORD=<supabase-password>
DATABASE_HOST=aws-0-ap-south-1.pooler.supabase.com
DATABASE_PORT=6543
```

Part 3: Docker Compose Files (Split by Platform)

Your current single `docker-compose.yml` runs everything locally. For production, we split it into **two files**.

3.1 `docker-compose.prod.iitk.yml` (For the IITK Server)

This runs **only** the ML service, Celery workers, Channels, and the Nginx load balancer on the IITK server:

```yaml
services:
  # ML Crowd Monitoring Service
  ml_service:
    build:
      context: ./ml_service
      dockerfile: Dockerfile
    container_name: upside_dine_ml
    command: uvicorn main:app --host 0.0.0.0 --port 8002
    ports:
      - "8002:8002"
    env_file:
      - .env.prod.iitk
    restart: always

  # Celery Worker (connects to Upstash Redis + Supabase DB)
  celery_worker:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: upside_dine_celery_worker
    command: celery -A config worker -l info
    env_file:
      - .env.prod.iitk
    restart: always

  # Celery Beat Scheduler
  celery_beat:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: upside_dine_celery_beat
    command: celery -A config beat -l info
    env_file:
      - .env.prod.iitk
    restart: always

  # Django Channels (WebSocket server)
```

```yaml
  channels:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: upside_dine_channels
    command: daphne -b 0.0.0.0 -p 8001 config.asgi:application
    ports:
      - "8001:8001"
    env_file:
      - .env.prod.iitk
    restart: always

  # Nginx Load Balancer
  nginx_lb:
    image: nginx:alpine
    container_name: upside_dine_lb
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.prod.conf:/etc/nginx/nginx.conf
    restart: always
```

3.2 Render & Koyeb (No Docker Compose needed)

These platforms **auto-build your Dockerfile** from the GitHub repo. You just point them to:

- **Root Directory:** `backend/`
- **Dockerfile Path:** `Dockerfile`
- **Start Command:** `sh -c "python manage.py migrate && python manage.py collectstatic --noinput && gunicorn config.wsgi:application --bind 0.0.0.0:8000"`

3.3 Netlify (No Docker at all)

Netlify builds static files directly:

- **Build Command:** `npm run build`
- **Publish Directory:** `dist`
- **Base Directory:** `frontend/`

Part 4: Nginx Load Balancer Config

`nginx/nginx.prod.conf` (On the IITK Server)

This is the key piece — it distributes API traffic across Render and Koyeb:

```
events {
    worker_connections 1024;
}

http {
    # Load balance between Render and Koyeb backend instances
    upstream api_backends {
```

```
        # Round-robin load balancing (default)
        server upside-dine-api.onrender.com:443;
        server upside-dine-api.koyeb.app:443;
    }

    upstream channels {
        server channels:8001;
    }

    server {
        listen 80;
        server_name <iitk-server-ip>;

        client_max_body_size 100M;

        # API requests → Load balanced to Render + Koyeb
        location /api/ {
            proxy_pass https://api_backends;
            proxy_set_header Host $proxy_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_ssl_server_name on;
        }

        # Django Admin → Load balanced too
        location /admin/ {
            proxy_pass https://api_backends;
            proxy_set_header Host $proxy_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_ssl_server_name on;
        }

        # WebSocket → Local Channels container
        location /ws/ {
            proxy_pass http://channels;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade";
            proxy_set_header Host $host;
        }

        # ML Health check
        location /ml/ {
            proxy_pass http://ml_service:8002;
            proxy_set_header Host $host;
        }
    }
}
```

Part 5: How It All Connects (Data Flow)

```
Student opens app in browser
         │
         ▼
   ┌──────────┐
   │  Netlify  │  (serves static React app)
   └──────────┘
        │ API calls to VITE_API_URL
        ▼
┌──────────────────┐
│ IITK Nginx (LB)  │  (port 80)
└──────────────────┘
   │         │
   ▼         ▼
┌──────┐  ┌──────┐
│Render│  │Koyeb │   (Django API — round robin)
└──────┘  └──────┘
   │         │
   ▼         ▼
┌──────────────────┐
│ Supabase (PgSQL) │   (shared database)
│ Upstash  (Redis) │   (shared cache/broker)
└──────────────────┘
```

Part 6: Deployment Checklist

1. ☐ Create Supabase project, copy DB credentials
2. ☐ Create Upstash Redis, copy connection URL
3. ☐ Deploy backend to Render (connect GitHub, set env vars, point to `backend/` dir)
4. ☐ Deploy backend to Koyeb (same setup, second load-balanced node)
5. ☐ Create `.env.prod.iitk` on IITK server with Supabase + Upstash creds
6. ☐ Create `docker-compose.prod.iitk.yml` and `nginx/nginx.prod.conf` on IITK server
7. ☐ Run `docker compose -f docker-compose.prod.iitk.yml up -d` on IITK server
8. ☐ Deploy frontend to Netlify (connect GitHub, set `VITE_API_URL` to IITK IP)
9. ☐ Update `CORS_ALLOWED_ORIGINS` on Render/Koyeb to include Netlify URL
10. ☐ Run `python manage.py createsuperuser` on Render to create admin account
11. ☐ Test end-to-end: Register → OTP → Login → API calls → ML feed